CS 230 Project Software Design Template



Draw it or lose it
CS 230 Project Software Design Template
Version 1.0

Table of Contents

CS 230 Project Software Design Template 1

Table of Contents 2
Document Revision History 2
Executive Summary 3
Requirements 3
Design Constraints 3
System Architecture View 3
Domain Model 3
Evaluation 4
Recommendations 5

Document Revision History

Version	Date	Author	Comments
1.0	09/23/2026	Ethan Goncalves	Initial software implementation and design for Draw it or Lose it
1.1	10/10/2026	Ethan Goncalves	Added evaluations

Instructions
Fill in all bracketed information on page one (the cover page), in the Document Revision History table, and below each header. Under each header, remove the bracketed prompt and write your own paragraph response covering the indicated information.

Executive Summary

The Gaming Room, as it currently stands, offers Draw it or Lose it as a Android application and has wants to expand the game into the web-based application ecosystem so it may be accessed across multiple platforms. This application will have to support multiple games, teams, along with players while having the option to hold unique team names and game names. In order to meet these requirements, the application will utilize object-oriented programming principles along with software design patterns to keep the code organized and reusable. GameService class will utilize the Singleton pattern so that one instance manages the games data. Iterator patterns will be used to search for existing games, teams, and players before new ones are created. Entity class will store shared information like ID and name. This design will provide The Gaming Room with a structured foundation for it to be developed as a web-based version of the game and can later support users on various devices and operating systems.

Requirements

The Gaming Room has the requirement for Draw it or Lose it to be expanded as its current Android format is not compatible with the requirement for a web-based application handling support across multiple platforms. The game must be able to hold one or more teams, and each of the teams must be able to also hold multiple players. Another requirement is the game and team names have to be unique so that users can select their own names or see what names are already in use. This application must also be able to assign identifiers to games, teams, and players that are unique to each. Additionally, only one instance of GameService class should exist in the memory base at a time allowing for game data to be managed consistently through the application.

Design Constraints

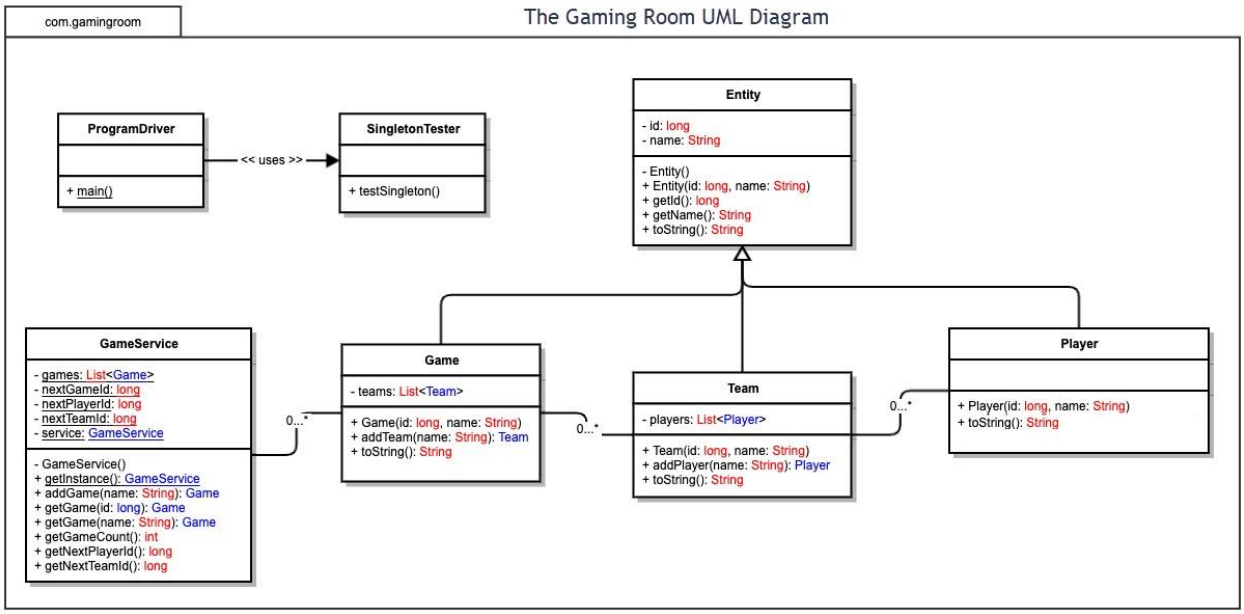
When reviewing the project one glaring design constraints for Draw it or Lose it was the application needing to work on a web based distributed environment on top of supporting users on different platforms and devices. Because of these constraints, the application should be designed so that the main game logic and data are consistently managed instead of depending on specific operating systems. This system must also support multiple games, teams, and players simultaneously while keeping games and team names unique. Unique identifiers for games, teams, and players are also a necessity so that each object can be managed correctly within the application. Another constraint is maintaining consistent application data for multiple users when they are accessing the system. The GameService class can help to address this by utilizing the Singleton pattern so that only one instance manages the game information for memory. The iterator pattern also supports our requirement for unique names by searching existing objects before the new ones are created. Since the application will operate over networks, developers must consider issues like connectivity, readability, and security for the application when it is expanded into a web based system.

System Architecture View

Please note: There is nothing required here for these projects, but this section serves as a reminder that describing the system and subsystem architecture present in the application, including physical components or tiers, may be required for other projects. A logical topology of the communication and storage aspects is also necessary to understand the overall architecture and should be provided.

Domain Model

The UML class diagram here shows the main classes in the Drae it or Lose it and the relationships to each other. Entity is the base class for the Game, Team, and Player, allowing them to share common attributes such as id and names through inheritance. GameService manages the games, each game can also contain multiple teams, and each team will contain multiple players. This design also utilizes encapsulation with keeping data within each class and being able to access it through methods. The Singleton pattern makes sure that only one GameServoce instance exists, with the iterator patter used to search collections and prevent duplicate names. These principles keep the application organized and easy to maintain.



Evaluation

Using your experience to evaluate the characteristics, advantages, and weaknesses of each operating platform (Linux, Mac, and Windows) as well as mobile devices, consider the requirements outlined below and articulate your findings for each. As you complete the table, keep in mind your client's requirements and look at the situation holistically, as it all has to work together.

In each cell, remove the bracketed prompt and write your own paragraph response covering the indicated information.

Development Requirements	Mac	Linux	Windows	Mobile Devices
Server Side	For MacOS it is capable of hosting a web based application like Draw it or Lose it utilizing server software like Apache Tomcat, which is capable of supporting Java applications. An advantage of MacOS is the Unix architecture and compatibility with java development tools. A drawback could come from Apples hardware typically being expensive and is not the most common for large scale server hosting compared to Linux. While MacOS does not typically need a separate OS license, hardware and hosting costs may make it less cost effective for The Gaming room if application expands to thousands of players	Linux can be a strong option in hosting Draw it or Lose it because of the support for web server software such as Apache HTTP server, Nginx, and Apache Tomcat for Java applications. Linux is commonly and widely utilized for hosting web applications for its reliability, customizability, and capability of handling large numbers of users with the right configuration. A second advantage comes from many Linux distributions, like Ubuntu server which is free and open source, which reduces operating system licensing costs. This being considered, Linux can also require more technical knowledge in configuring, maintaining, and securing. Linux also works well with cloud services and load balancing, allowing expansion for the game's expansion. These advantages make Linux a competitive and good choice for the server.	The Windows Server is one capable of hosting Draw it or Lose it using Microsoft's internet services or Apache Tomcat. One big advantage is the graphical interface and integration with Microsoft's management tools making administration easier for teams familiar with Windows. Windows servers can also offer enterprise security features along with many support options. One disadvantage though comes from the licensing costs because windows servers typically will need licenses. Windows servers can also need higher resource requirements depending upon configuration. Although windows has great support and utility the added expenses make it not as appealing of a choice even if it can handle thousands of users.	Mobile devices that are running Android or IOS would be designed around accessing the applications rather than hosting massive web servers. That being said mobile devices are able to run limited server software but would not be practical for hosting Draw it or Lose for thousands of players. Mobile devices do have drawbacks and limitations for processing resources, battery life, network connectivity, and operating system restrictions on background activity typically. Android and IOS generally do not require operating system licenses for device owners, and maintaining enough devices in hosting a large application would lead to inefficiencies. Instead of hosting the best setup would come from mobile devices communicating with dedicated servers managing game information, players, and scores. This would allow for mobile users to access the game while not putting a massive workload on their mobiles.

Client Side	In Mac user's cases, Draw it or Lose should likely be accessible with modern web browsers like chrome, Firefox, and Safari. Applications could use HTML5, JavaScript or CSS to build and create a responsive interface working without requiring installation of a separate program. Development considerations come from considering the game features and functionality are consistent across browsers which are programmed and work differently. Developers need to have experienced web development projects along with browser testing, and testing reposefulness. Browser based applications have reduced costs and time involved in creating this way not requiring separate macOS applications. This way of approaching the development will address and allow for Mac users sharing gameplay and access across operating systems.	Linux users for draw it or Lose it can access from supported web browsers like Firefox and Chrome. Developing this application could be done with HTML, JavaScript, and CSS allowing for interface to function across the multiple operating systems in order to save time not creating a separate Linux version. A caveat could come from Linux having many distributions, and accounting for the differences in that and browser configurations, fonts, and graphics can lead to a huge time investment and large amount of testing. Developers will also need to have knowledge on the web standards and experience in identifying any browser related problems. Utilizing a shared web application could reduce developmental costs and downtime although still requires a lot of time. Linux should be able to handle what is being asked of the developers.	Windows users would be able to connect to Draw it or Lose it through browsers like Chrome, Edge, or Firefox. Keeping with developing on a web interface would be fine with Windows being widely supported. Developer's tests would consist of tests across the various different browsers and configurations most of their users would have. JavaScript or HTML could be used to create the interface with the server handling games and communication between players. This approach can save time with code supporting multiple platforms, although browser test and updates would still require time and effort.	Undertaking supporting Mobile devices like Android and IOS will require Draw it or Lose it to have a responsive web interface that can adjust to smaller screen orientations and touch inputs. Developers would have to optimize for browsers that are popular and compatible with mobile like chrome and safari, but will also need to take into account the many different screen sizes and configurations of mobile devices . A route to make this happen on web instead of native applications could be utilizing HTML5, and JavaScript, which this approach could save money on development in this way. In terms of mobile development lots of time will be needed for testing, accessibility, and performance across devices. Developers would have to be experienced in responsive designs, and mobile browser behaviors to deliver a smooth consistent experience

Development Tools	Developers on mac can use HTML, CSS, JavaScript, and java to fully build Draw it or Lose it. The specific tools that could be used would be Visual Studio Code, Eclipse, and utilize Safari Web Inspector in supporting the development and testing. Most tools are free in the use case we need, but hardware is needed and most Mac hardware is expensive to buy into and used options are not as cheap in comparison to other brands. A specific Mac team could be unnecessary because it would be web based but testing would need to be done to determine consistency and bugs.	Linux supports the goal with access to Java for the back-end development and HTML, CSS and JavaScript, in building our interface for the game. Developers could use these various applications for development, Eclipse, Visual Studio Code, Apache Tomcat or Nginx. Most tools we can use will be free and open source giving us low barriers for entry in using Linux. Development team will need to be knowledgeable in Linux administration and server's configurations.	Windows has support for Java, CSS, HTML, and JavaScript, while tools like Visual Studio Code, and Apache could be our IDE. Most of the Developmental tools are free which is a great low barrier for entry in our case although Windows servers could require licensing tool fees. Developers would need to know and understand Windows administration and web development.	Mobile development In the games creation we have access to CSS, HTML, and JavaScript can all support our Android and IOS browser development. IDE's available to us are Visual Studio Code, Xcode, and Android studio which can all contribute to our development and testing. These tools are mostly free although we would need Apple hardware which can be costly. Developers should be knowledgeable in responsive design and mobile testing.

Recommendations

Analyze the characteristics of and techniques specific to various systems architectures and make a recommendation to The Gaming Room. Specifically, address the following:

1. **Operating Platform**: In examining a suitable operating platform for The Gaming room, I am recommending using Linux, in particular a Ubuntu Server, to host a web-based version of Draw it or Lose it. Linux became a strong recommendation because of its reliability, cost effectiveness, and capability of large user number support. Additionally choosing an Ubuntu Server has major upside of being open source, helping to reduce licensing expenses in comparison of competition like Windows servers. Another factor that played into this selection was Linux supporting Java applications from software like Apache Tomcat allowing for the game to operate within a centralized server so users can access it through web browsers on other operating systems like macOS, Windows, mobile devices, and Linux distributions. All of these facts taken into consideration allow for The Gaming Room to expand its application without having to develop separate versions for each and every operating system they want to be playable on.

2. **Operating Systems Architectures**: Ubuntu Server for our needs offers Linux-based operating system architecture specifically designed around stability, security, and efficient management of the systems resources. Linux utilizes a kernel that has modular components that manage the process, memory consumption, hardware, and file systems. Operating system for Linux also separates kernel space from user space which is helpful in preventing applications from directly interfering in critical system functions. Advantages created from Linux additionally being support for multitasking allowing for multiple process and applications running simultaneously. This would be important for our use in Draw it or Lose it with allowing the server to handle multiple games, teams, and players simultaneously. Ubuntu Server also allows support on different hardware architectures, like x86-64 and ARM64, which provides choice and flexibility when considering server hardware. Another aspect considering the client-server architecture can allow main game logic and data to be on the Linux server and users access the application from various operating systems leading to not needed separate game versions.

3. **Storage Management**: In Draw it or Lose it, I recommend the use of Ubuntu's Server ext4 file system and a database like PostgreSQL in managing our applications storage needs. Ext4 file system is reliable and can support a larger amount of data which allows Linux to organize files and manage the storage efficiently. PostgreSQL can store the persistent data needed like player accounts, game sessions, teams, and any unique identifiers that will need to go to long term memory storage. Using this database can help maintain data consistency and prevent duplicate records that shouldn't be there. Some important considerations would be to ensure game information remains available when needed and protected. Ubuntu Server supports file permissions, automated backups, and storage monitoring, all helping to prevent unauthorized access and data integrity. Game images along with the application files can be stored sperate from the database which can improve organization of data and performance. This can address future needs as Gaming Room expands, additional storage capacity or cloud-based solutions can be implemented and accommodate future users while having reliable access to game data.

4. **Memory Management**: Ubuntu Server utilizes several memory management techniques in efficiently allocating system resources where need be and supports multiple applications running simultaneously. A specific standout technique is virtual memory, allowing for processes

to use their own virtual address spaces with the operating system managing how memory is allocated to the physical RAM. Linux also is capable of paging to manage memory and can utilize swap space when the RAM is limited. Taking full advantage of these techniques helps to prevent any individual application from interfering with one or another while keeping system stability. In our use case, for Draw it or Lose it having efficient memory management is crucial because of the expectation of having multiple games, teams, and players active at the same time. If we pursue Java based applications for the backend, we can also take advantage of Java's garbage collection which automatically reclaims memory from objects that are not in use. Ubuntu server can also take advantage of memory caching in improving performance by keeping frequently accessed data available. In combination of Linux's memory management and Javas garbage collection , we can expect the application to have great performance and reduce memory related issues as we increase user count.

5. **Distributed Systems and Networks**: In analyzing a system and setup that accommodates the needs for Draw it or Lose it I recommend using a client server architecture that allows users on Windows, macOS, Linux, Androids, and IOS connection to a centralized Ubuntu Server. This would allow for players to access the game through web browsers, and communication with the server over internet on HTTPS and TCP and IP protocols. Server would handle main game logic, players information, and the game sessions while clients are displayed game interface. Distributed approach allows users on different platforms to participate in same games while not requiring separate applications for every available operating system. Important considerations come from maintaining reliable communication between different components in the system. Specifics come from network outages, server failures, or large traffic volumes all could contribute to interruptions of active games or players' information being down. Reducing these risks will be crucial and there is fixes especially if a Load balancing unit is implemented, or server redundancies and data backups. Having a shared database and centralized session management can help maintain a smooth consistent experience for users. These specific solutions can improve availability, performance, and reliability for The Gaming Room.

6. **Security**: In taking a step back and examining a major consideration of security we need to understand how security affects Draw it or Lose it especially if the goal is a large user base across multiple platforms. Ubuntu servers have some built In security features, which include user permissions, access controls, and firewall management. The Gaming room can take advantage of the built-in features and use the Firewall available to them from Ubuntu and restrict unauthorized network traffic and implement limits and protections on necessary services. Preforming regular security updates and checks along with security monitoring will also add a layer to security and help mitigate risks from vulnerabilities known or unknown to The Gaming Room. Protecting user information is a top priority and to implement that protection I would recommend utilizing HTTPS and TLS encryption to make sure secure communication is established between players and the server. User accounts are an additional security concern, making sure to implement strong password requirements and role-based access controls is a priority requirement. Lastly database permissions and encryption at rest will help to protect the stored player information and will maintain a practical and efficient security posture for The Gaming Room